Kolam Studio ��� README

Kolam Studio

Reading the hidden mathematics of a vanishing art —
identify it, recreate it exactly, and generate it anew.

🔗 **Live demo** · **SIH12507** · AICTE · Heritage & Culture

Overview

A **kolam** is a South Indian threshold art drawn each morning in rice flour: a grid of dots (*pulli*), a line looping around them, symmetry, and — in *sikku kolam* — one continuous unbroken loop. The art survives in photos, but the **rules** that construct it are fading.

Kolam Studio teaches a machine to read those rules. Give it any kolam and it detects the design principles, recreates the kolam **exactly**, and generates brand-new original patterns — all **in the browser**, offline-ready.

What it does

- 🔍 **Identify** — analyses a kolam image *in the browser*: dot grid (pulli), symmetry type and structure. Non-kolam images are rejected.
- ✏ **Recreate** — reconstructs the uploaded kolam **cell-by-cell** from inferred tile orientations (exact on clean images), animated stroke-by-stroke with a glowing pen-head.
- ✨ **Generate** — endless new, original kolams under the same rules:
  - **Single unbroken line** — the whole kolam as one continuous closed loop.
  - **Custom dot layouts** — Square / Diamond / Plus presets + a click-to-toggle dot editor.
  - **SVG & PNG export** for crisp, print-ready output.

How it works

```
Kolam image  →  In-browser analyzer  →  Design principles  →  Tile generator
 (upload)        dots · symmetry · tiles    (grid, symmetry)     (closed-loop canvas)
```

Under the hood, each grid cell holds one of two curved tiles that join into guaranteed **closed loops** around the dots (a small *tile-grammar*). A loop-merging solver can collapse the whole kolam into a **single continuous line**. The analyzer detects the dot grid by colour + geometry, classifies symmetry by image self-similarity, and infers each cell's tile to reproduce the kolam faithfully.

Tech stack

Layer	Technology
Frontend / app	Vanilla **JavaScript**, HTML5 Canvas, CSS
Analyzer (shipped)	Vanilla JS — runs fully client-side, no backend
Reference analyzer	**Python + OpenCV + NumPy**
Server (optional)	**Flask** (serves the app + reference `/analyze`)
Delivery	Installable **PWA** (offline), deployed as a static site
Quality	**pytest** suite + GitHub Actions **CI**

Run locally

No dependencies — it's a static site:

```
python -m http.server 8000     # from this folder → http://localhost:8000
```

Optional Flask server with the Python reference analyzer:

```
pip install -r requirements.txt
python app.py                  # → http://localhost:5000
```

Test

```
pip install pytest && pytest -q     # 11 tests — analyzer, endpoints, reconstruction
```

Deploy

The folder is fully static — deploy anywhere (Render Static Site / Netlify / Vercel / GitHub Pages). No server means **no cold-start**, so a live demo can't time out.

Project structure

```
index.html              static app (root)
static/                  analyzer.js · kolam.js · style.css · logo-mark.svg
kolamgen.py              reference generator (closed loops + single-loop solver)
analyzer/                Python reference analyzer (detect · symmetry · reconstruct)
app.py                   optional Flask server
data/samples/            sample kolams + a blank negative + samples.json
tests/                   pytest suite     .github/workflows/  CI
manifest.webmanifest · sw.js · icons      PWA
```

Highlights

- Exact per-cell **reconstruction** (100% tile accuracy on the clean test set)
- **Single unbroken line** generation (traditional sikku ideal)
- **Custom dot layouts** (diamond, plus, freeform)
- Runs **fully in-browser**, installable & **offline**
- **Real-photo fallback** (illumination-normalise + Otsu + line-period estimate)
- Tests + CI; honest, documented limitations

Limitations & roadmap

Coloured-dot / digital kolams are read exactly. On plain photographs the grid is *approximate* (symmetry still reliable); sharper photo handling (RANSAC grid-fit, perspective correction), regional styles, and motif-learning are next.

Team Kolam · Smart India Hackathon 2025 · SIH12507
kolam-studio.onrender.com